Steam Data Pipeline Implementation Using Apache Spark (M3)

Keyan Luo
Kennesaw State University
Kennesaw, Georgia, USA
Email: kluo2@students.kennesaw.edu

***Abstract*—This project implements a complete Big Data pipeline using Apache Spark. The system processes a large-scale Steam dataset with over 41 million records. The pipeline integrates multiple data sources, performs data cleaning and transformation, and produces aggregated analytical results. All stages of the pipeline run automatically from ingestion to output generation.**

I. Implementation Summary

In this milestone, I built a complete data pipeline using Apache Spark. The pipeline processes a large dataset with more than 41 million records. It includes several steps such as data loading, processing, aggregation, and output.

I separated the pipeline into multiple Python files, so each part has a clear role. For example, ingestion is used for loading data, processing handles cleaning and transformation, aggregation computes the final results, and storage saves the output.

The pipeline can run from start to finish without manual work. After running the script, the final result is saved as a CSV file. Overall, the system works successfully and produces meaningful results.

II. Data Flow Description

The pipeline starts by loading raw data from multiple files in the data folder. These include games data, users data, metadata, and recommendations.

After loading the data, the pipeline performs cleaning and preparation. Some missing values are removed based on important columns, so invalid data will not affect the results.

Then, the datasets are joined together using the common key app_id. This allows data from different sources to be combined into one dataset.

Next, the pipeline performs aggregation. It calculates the average recommendation score and the number of reviews for each game.

Finally, the processed data is written to the output folder as a CSV file. The whole pipeline runs automatically without manual steps.

III. Transformation Decisions and Rationale

During the pipeline design, several decisions were made to improve data quality.

First, missing values were handled selectively. Instead of removing all rows with missing data, only rows missing important fields such as app_id or recommendation values were removed. This helps keep more useful data.

Second, joins were used to combine datasets. In some cases, left join was used instead of inner join, so more records can be preserved.

Third, a new column was created to convert recommendation values into numeric form. This allows aggregation functions to work correctly.

After that, aggregation was applied to compute the average recommendation score and total number of reviews for each game.

For the output stage, I used a Python-based method to write the final result after aggregation. This is because I encountered compatibility issues when trying to use Spark's default write function in a Windows environment. Since the data size becomes much smaller after aggregation, this method works well and remains efficient.

IV. Challenges and Solutions

One of the main challenges was configuring Spark and Hadoop on Windows. When trying to use Spark's built-in write function, errors related to Hadoop libraries occurred.

After trying several solutions, I decided to use a simpler output method. Instead of using Spark to write the data, I collected the aggregated results and wrote them using Python. This avoided environment issues and allowed the pipeline to run successfully.

Another challenge was handling a large dataset. Processing more than 41 million records required careful design of joins and aggregation steps to make sure the system runs efficiently.

These challenges helped me better understand how practical data pipelines work.

V. Performance Observations

The pipeline successfully processes a large dataset using Spark. Compared to single-machine tools, Spark makes it possible to handle this amount of data more efficiently.

The join and aggregation stages take the most time due to the dataset size. However, the pipeline still completes successfully.

After aggregation, the data size is reduced significantly, which makes the final output stage much faster.

VI. Output and Execution Results

Figure 1 shows the successful execution of the pipeline.

Figure 2 shows a sample of the output data after aggregation.

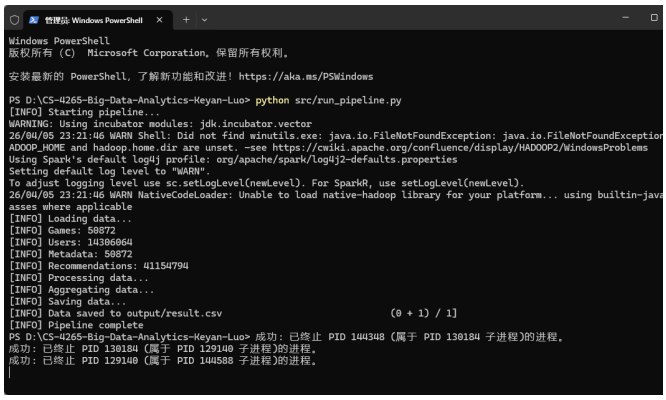

Fig. 1. Pipeline execution result in terminal

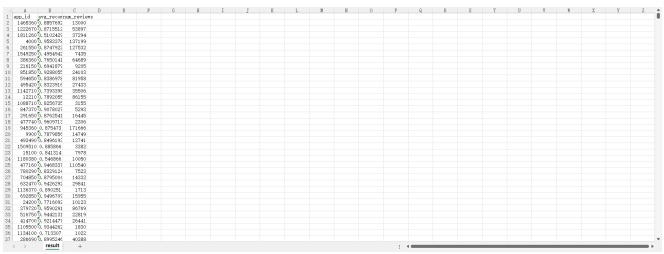

Fig. 2. Sample output data after aggregation

VII. Remaining Work for M4

In the next milestone, I plan to improve the pipeline further.

One possible improvement is to use more efficient storage formats such as Parquet.

I also want to improve the pipeline structure and make it more flexible. For example, it can be easier to change input paths or add new data sources.

Finally, I will improve documentation and make the system easier to understand.

VIII. Conclusion

This project implements a complete data pipeline using Apache Spark. The system processes large-scale data, performs distributed computation, and produces useful analytical results.

The pipeline demonstrates important concepts such as data integration, transformation, and aggregation. It also shows how to handle practical issues when working with real-world environments.